

can then return it to the client a little bit at a time. The program that generates the page need not remain open, thus releasing server resources during the possibly extended time the client requires to complete the transfer.

8. Reverse proxies can operate whenever multiple Web servers need to be accessible via a single public IP address. The Web servers listen on different ports in the same machine, with the same local IP address or, possibly, on different machines and different local IP addresses altogether. The reverse proxy analyses each incoming request and delivers it to the right server within the local area network.
9. Reverse proxies can perform multiple other forms of testing without placing JavaScript tags or code into pages.
10. The reverse proxy concept is also used in search engine marketing to automatically embed a destination website with usage tracking code that can be used for campaign reporting or campaign optimisation. This is typically accepted as being bad practice.

Let us now go on to achieve Points 4 and 8 from the above list.

So our objective is to primarily configure Apache such that it can cater to multiple application servers with only one public IP address. Also, we will look at how to rewrite URLs so that the client doesn't know about any internal server.

In this article, I will consider Fedora as the operating system installed on my server. Similar functionality can be achieved with any of the OSs that are supported by Apache. First, to install Apache HTTP server, run the following command either as the root user, or run the command with *sudo*:

```
yum install httpd
```

Based on the host OS, it will depend whether the 'yum' command has to be used or not. There are binaries available for direct installation as well. 'httpd' basically stands for Hypertext Transfer Protocol Daemon. Since the Apache Web server works on HTTP standards and the server always runs as a daemon (a background process), the package was named as *httpd*.

Now that the server is installed, let's just get it going:

```
/etc/init.d/httpd start
## OR ##
service httpd start
```

The default port of Apache is 80. This can be found in the *httpd.conf* file inside the Apache folder. On a Linux system, simply browse to the following location:

```
cd /etc/httpd/conf
```

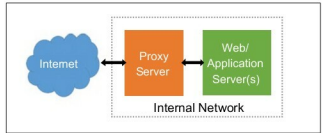


Figure 1: Selecting the interface list for packet analysis

Here, you will see a file called *httpd.conf*. This file is where all the magic happens. If you are working with a different OS, the file's location will be different, but essentially you need to search for this file only: *httpd.conf*.

First, let's check which default port has been listed inside this *conf* file. Look for a line similar to the following:

Listen 80

This is the default port on which Apache server is listening. Based on which port is open for your public IP, you can simply change this port number here. For example, if your port number is 6500, change the number next to 'Listen':

Listen 6500

Next, to check if the server is actually up and running, type the following command in the terminal:

```
netstat -tulpn | grep :portNumber
```

...where *portNumber* is the port number that you have set in the *httpd.conf* file. If the server is running, you will get an output as shown below:

```
tcp6      0 :::6500          :::*
LISTEN    61974/httpd
```

...where 6500 is the *port number* and 61974 is the PID (process ID) for the *httpd* daemon.

Now that the server is up and running, let's do the reverse proxy and URL rewriting configuration. First, ensure that the following line is not commented in the *conf* file:

```
Include conf.modules.d/*.conf
```

Next, we need to create a new module tag for the *mod_rewrite.c* module. Preferably, let's add this at the bottom of the file so that in case we face an error, we know where to look for it:

```
<IfModule mod_rewrite.c>
    RewriteEngine On
</IfModule>
```